

Abstract

As embedded systems operate closer to physical environments, fault tolerance becomes a critical design requirement. This report explores fault injection on a Linux-based RISC-V system emulated with QEMU. The study focuses on three fault types: bit flips, stuck-at faults, and instruction corruption. These faults reflect real-world issues, such as radiation effects and hardware wear-out. Python scripts control GDB through its machine interface to inject these faults at runtime. The scripts change CPU registers and instruction memory during program execution. Each event logs relevant data into structured DOCX reports. The virtual setup removes the need for physical hardware, making testing reproducible and cost-effective. This system allows engineers to record and observe the effects of injected faults. It also supports future work on automated fault analysis and digital resilience. The framework is modular and can adapt to different processor architectures. Its design encourages testing across diverse use cases. Researchers can extend it to simulate other fault models. This report offers a scalable and hardware-independent approach to fault injection in embedded platforms.

1. Introduction:

Fault Injection refers to the deliberate introduction of faults into a system to observe how it reacts and what warnings pop up in response to those faults (Koopman, 2016). It allows developers to observe failure modes, study system resilience, and measure how well the fault-handling mechanisms work. Fault injection offers insights into weaknesses that may go unnoticed under standard testing procedures, especially in safety-critical or resource-constrained systems. As Edge AI becomes more prevalent, more systems operate locally instead of relying on centralized, cloud-based servers (Li et al., 2020). These systems often function in unpredictable real-world conditions, such as fluctuating temperatures, exposure to electromagnetic interference, and the potential mishandling of sensitive hardware. In this context, fault tolerance is essential as it becomes a safety measure to ensure long-term functionality.

This project focuses on implementing fault-injection capabilities into QEMU. QEMU allows users to emulate RISC-V architectures, CPUs, and other peripheral devices on a host machine without requiring physical hardware. QEMU uses Dynamic Binary Translation (DBT) to translate the target machine's machine code into an instruction set that the host machine can execute. QEMU can emulate low-level behaviors like memory access, register states, and the instruction execution pipeline (Bellard, 2005). QEMU uses Just-In-Time (JIT) compilers to translate the target's machine code just before the host machine executes it. As a result, QEMU can run efficient and cycle-accurate simulations without hardware dependency.

This report aims to explore modifying QEMU to introduce faults at all three levels: the CPU, memory, and instruction pipeline. This report uses separate python scripts to simulate each type of fault injection. Using a separate file for each fault injection makes it easier to log, read, debug, and extend. These scripts will define fault models that emulate real-world errors, including bit flips, stuck-at fault, and instruction corruption. This report aims to perform these experiments on an RISC-V virtual machine running inside QEMU and inject faults at the instruction and register levels.

2. Fault Types in Digital Systems

Digital Systems are susceptible to various faults that can lead to system compromise, reliability, and performance issues. The first step in designing a robust and fault-tolerant system is to understand the types of faults the system will face in real-world operation. Understanding which and how faults may occur is the most crucial step in implementing effective and efficient fault injection strategies.

2.1 Types of Faults

A system can encounter three types of faults during operation. Any system may experience one or more of these faults at once.

2.1.1 Transient Faults (Soft Errors)

Transient faults refer to temporary disruptions in a system resulting from changes in external environmental factors. Such faults may or may not permanently damage the hardware, but they can temporarily change system behavior. Fluctuations or exposure to cosmic radiation are a primary cause behind transient faults. Cosmic radiation can induce Single-Event Upsets (SEU) by changing the charge state in semiconductor devices. Changes in the electromagnetic field around the device can also induce SEU (Ziegler and Lanford, 1979).

The physical environment around the system influences how frequently the system may experience transient faults. For example, a system operating at a high altitude is more exposed to cosmic radiation and may experience transient faults more frequently (Ziegler and Lanford, 1979). Similarly, any system operating around high-voltage machinery may experience transient faults resulting from electromagnetic-field interferences.

In most cases, transient faults do not require human intervention and disappear once the disturbances in the physical environment disappear.

2.1.2 Permanent Faults (Hard Errors)

Permanent faults result from physical defects, damage, and/or manufacturing imperfections. Permanent faults are persistent and require human intervention. In most cases, the faulty hardware needs repair or replacement. Common causes include usual wear and tear during operation and defects induced during fabrication (Abramovici et al., 1994).

Stuck-at fault is an example of a permanent fault. It usually occurs when a signal line continuously shows either a logical high or a logical low value, regardless of the intended operation. Permanent faults can severely impact system functionalities and are challenging to detect without structural testing methodologies (Abramovici et al., 1994).

2.1.3 Intermittent Faults

Intermittent faults are sporadic and unpredictable errors that occur under specific conditions, such as voltage fluctuations, temperature variations, or mechanical stress. Unlike transient faults, intermittent faults may recur and are often challenging to diagnose due to their non-deterministic nature. For example, voltage fluctuations can lead to unstable power supply levels, causing components to behave erratically (Qi et al., 2014).

These faults are particularly problematic because they can mimic both transient and permanent faults, making them difficult to isolate and rectify. Effective mitigation requires comprehensive monitoring and adaptive fault-tolerant designs that can accommodate such unpredictable behaviors (Qi et al., 2014).

2.2 Fault Models

2.2.1 Bit Flips

Bit flip is one of the most common soft errors. A single bit changes its state from 0 to 1 or 1 to 0 in memory, register, or instruction code. Bit flips simulate transient faults resulting from radiation, noise, or electromagnetic interference (Ziegler and Lanford, 1979).

2.2.2 Stuck-at Faults

These faults lock a signal line to a constant value (logical high or low). Developers use stuck-at faults to simulate permanent faults and hardware verification (Abramovici et al., 1994).

2.2.3 Timing Errors and Pipeline Disruption

Fault injection introduces artificial delays or reorders instructions in the pipeline to disrupt execution flow. These faults expose data hazards and timing issues, especially in RISC architectures with tightly synchronized stages. Even minimal timing mismatches can trigger execution failures (Roberts, 1999).

2.2.4 Instruction Corruption

Instruction corruption changes the opcode or operand during the fetch or decode stage. These faults simulate software corruption or register decode errors and test the system's ability to handle malformed instructions under real-time conditions (Roberts, 1999).

2.3 Why Use QEMU for Fault Injection

QEMU offers a solid platform for fault injection for several key reasons:

- Its open-source design gives direct access to CPU execution, memory access, and device interfaces.
- It natively supports RISC-V, including both RV32 and RV64 instruction sets.
- It uses dynamic binary translation (DBT) to simulate realistic execution flow and timing under fault conditions.
- It eliminates the need for physical hardware, enabling reproducible and controlled testing in software.

- Researchers and engineers have used QEMU in fault injection tools like FAIL* and QEMU-Injector, proving its value as a lightweight yet powerful testbed.

Researchers and engineers have leveraged QEMU in fault injection tools like FAIL* and QEMU-Injector, demonstrating its value as a lightweight yet powerful testbed for fault tolerance studies (Koppenstein, 2020).

3. Methodology

3.1 Prerequisites

Before running any fault injection scripts, ensure the following:

3.1.1 Hardware & Environment:

- Windows 11 Pro with WSL2 (Ubuntu 24.04.5 LTS).
- Python 3.12 virtual environment with necessary packages installed:
 - pygdbmi
 - python-docx
- QEMU 10.0.50 or later installed with RISC-V support.

3.1.2 Kernel and Disk Images:

- Pre-built RISC-V Linux kernel image at:
~/kernel-build/linux-6.14.5/arch/riscv/boot/Image.gz
- Pre-installed Ubuntu RISC-V image at:
~/ubuntu-24.04.2-preinstalled-server-riscv64+nezha.img

3.1.3 Directories on Windows (mounted inside WSL):

- F:\loggings_bit_flip
- F:\loggings_stuck_at
- F:\loggings_instruction_corruption

These folders store QEMU serial logs and fault injection reports.

3.2 Fault Injection

This study uses automated Python scripts to inject faults at the register and instruction levels. The faults target a RISC-V virtual machine operating under QEMU. Each script activates a specific fault model and generates a report for later analysis. The system runs all scripts inside an isolated Python virtual environment configured for GDB automation. This setup ensures consistency across multiple test iterations.

The experiment requires a QEMU instance running a Linux-based RISC-V guest operating system. The system launches QEMU with GDB stub support, which enables external control through port 1234. After starting QEMU with the guest image, the Python script connects to the GDB stub. It uses the pygdbmi library to establish machine interface (MI) communication between GDB and Python.

The bit-flip injection script toggles a specific bit in RISC-V general-purpose registers. The script pauses the virtual CPU and reads the register's current value during each iteration. It applies XOR logic to flip the target bit and writes the new value. After completing the bit flip, it resumes execution and waits briefly before the next cycle. Each iteration logs the original and altered values in a DOCX report. The system saves this report to a predefined directory at the end of testing.

The stuck-at fault injection script simulates permanent faults by forcing specific bits to fixed values. The script operates in two phases: one holds bit 3 at logical '1'; the other, at logical '0'. In each phase, it accesses each target register and applies bitwise OR or AND masking. The script then writes the updated value back into the register. It records register values before and after modification in a DOCX file. To maintain stability, it temporarily suspends execution during register access. Like the bit-flip script, it runs a fixed number of iterations and stores its report in a designated directory.

The instruction corruption script modifies executable memory by directly altering instruction words. It first extracts symbol information from a precompiled vmlinux file. Then it calculates the kernel base address and traverses a target instruction region using a fixed stride. The script reads 32-bit words using GDB commands, applies a predefined XOR mask, and writes the modified word back. This process repeats for a fixed number of instructions. The script documents

both original and altered opcodes with their corresponding addresses. It also handles connection retries and ensures a valid GDB session before proceeding.

All three scripts generate Word documents summarizing the fault injection process. These include affected addresses or registers and related bit-level changes. The reports act as the primary artifacts for analyzing the fault injection mechanisms. They do not evaluate system-level behavior or responses to injected faults. Each script runs independently and is launched using `python <script_name>.py` after activating the virtual environment with source `~/qemu-fault-env/bin/activate`.

These fault injection methods offer a structured approach to injecting faults at various abstraction levels. They simulate transient and permanent faults typically found in physical systems. The process does not require modifying the guest kernel or accessing hardware directly.

3.2.1 Bit Flip Fault Injection

The bit-flip script starts by launching GDB in MI3 mode and establishing a connection to the QEMU GDB stub. It targets a set of general-purpose RISC-V registers and proceeds through a loop of controlled fault injections. GDB halts the virtual CPU to inspect the register state during each iteration. The script then reads the current register value using the `-data-evaluate-expression` command. It flips bit 3 through a bitwise XOR operation and writes the modified value back using the `set $reg = value` command. Each modification, including the original and altered values, is logged in a Word document. This process emulates transient faults resulting from cosmic radiation or electromagnetic-field fluctuations, which momentarily disrupt bit-level integrity in processor registers.

3.2.1.1 QEMU Startup

```
qemu-system-riscv64 -machine virt -cpu rv64 \  
-kernel ~/kernel-build/linux-6.14.5/arch/riscv/boot/Image.gz \  
-drive file=~/ubuntu-24.04.2-preinstalled-server-riscv64+nezha.img,format=raw,if=virtio \  
-append "root=/dev/vda1 console=ttyS0" \  
-serial file:/mnt/f/loggins/qemu_serial_output.log \  
-gdb tcp::1234 -S
```

3.2.1.2 Run Script

```
source ~/qemu-fault-env/bin/activate  
python bit_flip_fault.py
```

3.2.1.3 Output

- Report saved at:
F:\loggins\bit_flip_fault_report.docx
- Serial logs at:
F:\loggins_bit_flip\qemu_serial_output.log

3.2.2 Stuck-At Fault Injection

The stuck-at fault injection script introduces permanent faults by forcing bit 3 into fixed states. Like the bit-flip script, it starts by launching GDB in MI3 mode and connecting to QEMU. It targets the same general-purpose RISC-V registers and runs separate phases for each fault type. During the stuck-at-1 phase, it sets bit 3 to logical high using a bitwise OR mask. During the stuck-at-0 phase, it clears bit 3 using an inverted bitwise AND mask. During each iteration, GDB halts the virtual CPU to ensure safe access. The script reads the register value using the `-data-evaluate-expression` command. It then applies the mask and writes back the modified value using `set $reg = value`. The system logs original and altered values in a DOCX report for later analysis. This technique simulates permanent hardware faults caused by aging, damage, or fabrication defects.

3.2.2.1 QEMU Startup

```
qemu-system-riscv64 -machine virt -cpu rv64 \  
-kernel ~/kernel-build/linux-6.14.5/arch/riscv/boot/Image.gz \  
-drive file=~/ubuntu-24.04.2-preinstalled-server-riscv64+nezha.img,format=raw,if=virtio \  
-append "root=/dev/vda1 console=ttyS0" \  
-serial file:/mnt/f/loggins/qemu_serial_output.log \  
-gdb tcp::1234 -S
```

```
-append "root=/dev/vda1 console=ttyS0" \  
-serial file:/mnt/f/loggins_stuck_at/qemu_serial_output.log \  
-gdb tcp::1234 -S
```

3.2.2.2 Run Script

```
source ~/qemu-fault-env/bin/activate  
python stuck_at_fault.py
```

3.2.2.3 Output

- Report saved at:

F:\loggins_stuck_at\stuck_at_fault_report.docx

- Serial logs at:

F:\loggins_stuck_at\qemu_serial_output.log

3.2.3 Instruction Corruption Fault Injection

The instruction corruption script simulates faults by directly modifying executable memory within the instruction space. It begins by connecting to the QEMU GDB stub using a retry loop to ensure availability. The script then calculates target addresses starting from a fixed base and increments by four bytes. This stride matches the 32-bit alignment of RISC-V instructions. The script retrieves the instruction using the `x /1wx` GDB command. It applies a predefined XOR-based flip mask to simulate corruption in the instruction bits for each address. The script writes the altered instruction to memory using the `set address = value` command. The script repeats this for a fixed number of instructions in the selected range. It logs each original and modified instruction in a DOCX file for analysis. This method models instruction-level corruption from transient faults or bugs in memory fetching or decoding logic.

3.2.3.1 QEMU Startup

```
qemu-system-riscv64 -machine virt -cpu rv64 \  
-kernel ~/kernel-build/linux-6.14.5/arch/riscv/boot/Image.gz \  
-drive file=~/.ubuntu-24.04.2-preinstalled-server-riscv64+nezha.img,format=raw,if=virtio \  
-append "root=/dev/vda1 console=ttyS0" \  
-serial file:/mnt/f/loggins_instruction_corruption/qemu_serial_output.log \  
-gdb tcp::1234 -S
```

3.2.3.2 Run Script

```
source ~/qemu-fault-env/bin/activate  
python instruction_corruption_fault.py
```

3.2.3.3 Output

- Report saved at:

F:\loggins_instruction_corruption\instruction_corruption_report.docx

- Serial logs at:

F:\loggins_instruction_corruption\qemu_serial_output.log

4. Results

The fault injection experiments on the QEMU-simulated RISC-V kernel used three models: Bit Flip, Stuck-at, and Instruction Corruption. Each method targeted CPU registers or kernel instruction memory using Python-GDB automation. The system monitored outcomes for each case in a controlled and repeatable manner.

The Bit Flip model ran 50 cycles across registers x10, x11, and x12. The script flipped bit 3 while the script paused the virtual CPU during each cycle. Execution resumed without exceptions in all cases. The system maintained full

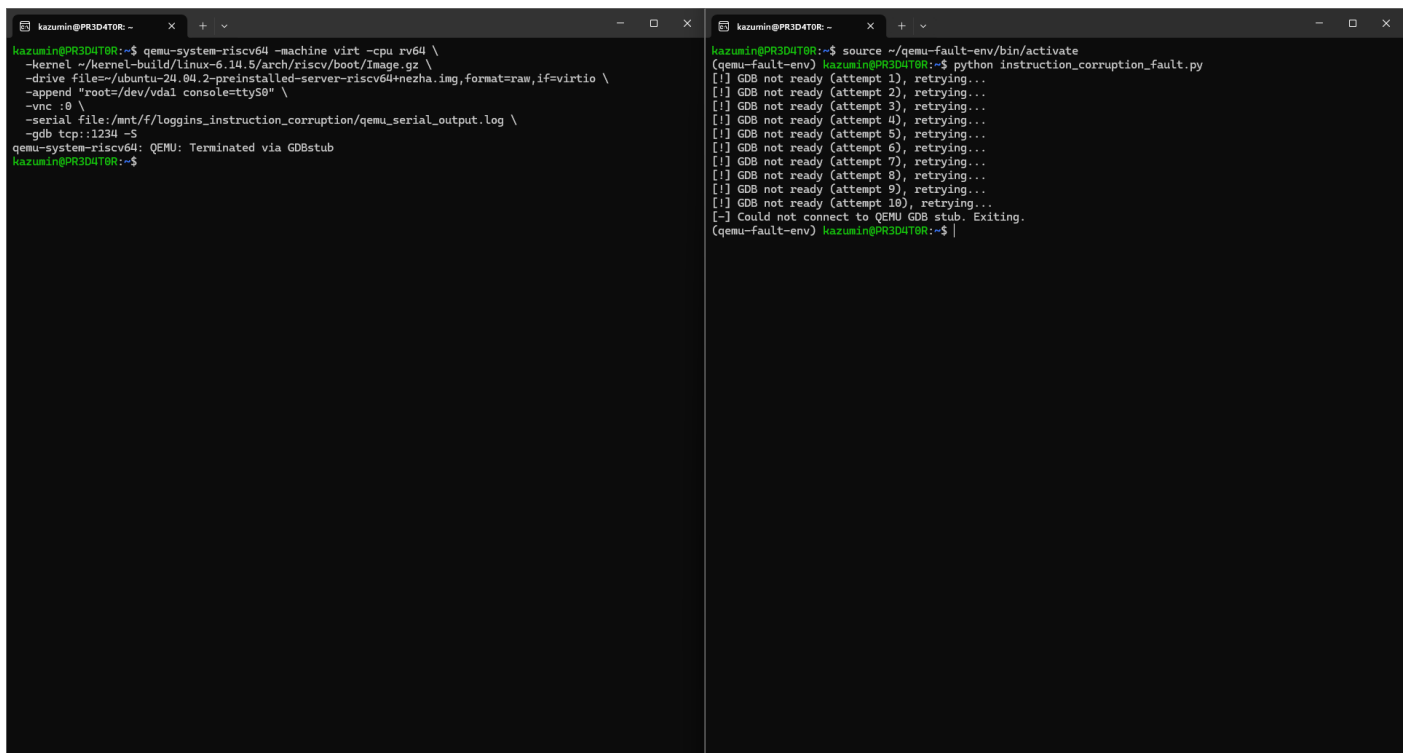
functionality, showing no behavioral changes. Logs confirmed each bit-flip through consistent register state updates. The complete data appears in Annex.

The Stuck-at model injected stuck-at-1 and stuck-at-0 faults into the same register set. Each injection forced bit 3 to a fixed value, either high or low, before resuming execution. Despite repeated faults, the system remained stable and uninterrupted. Post-injection logs confirmed fault application. Annex includes all results from the 50 test cycles.

The Instruction Corruption model attempted to flip bit 4 of a kernel instruction in memory. After resuming execution, QEMU terminated the session and displayed:

QEMU: Terminated via GDBstub

This crash occurred during the first fault cycle and halted all further testing. The system failed to continue due to the early termination. Figure 1 shows the error as captured from the screen.



The image shows two terminal windows side-by-side. The left window shows the QEMU command line for starting a RISC-V virtual machine. The right window shows the output of a script that attempts to connect to the QEMU GDBstub, resulting in a 'Could not connect to QEMU GDB stub. Exiting.' error.

```
kazumin@PR3D4T0R:~$ qemu-system-riscv64 -machine virt -cpu rv64 \
-kernel ~/kernel-build/linux-6.14.5/arch/riscv/boot/Image.gz \
-drive file=/ubuntu-24.04.2-preinstalled-server-riscv64-nezha.img,format=raw,if=virtio \
-append "root=/dev/vda1 console=ttyS0" \
-vnc :0 \
-serial file:/mnt/f/loggins_instruction_corruption/qemu_serial_output.log \
-gdb tcp:1234 -S
qemu-system-riscv64: QEMU: Terminated via GDBstub
kazumin@PR3D4T0R:~$

kazumin@PR3D4T0R:~$ source ~/qemu-fault-env/bin/activate
(qemu-fault-env) kazumin@PR3D4T0R:~$ python instruction_corruption_fault.py
[!] GDB not ready (attempt 1), retrying...
[!] GDB not ready (attempt 2), retrying...
[!] GDB not ready (attempt 3), retrying...
[!] GDB not ready (attempt 4), retrying...
[!] GDB not ready (attempt 5), retrying...
[!] GDB not ready (attempt 6), retrying...
[!] GDB not ready (attempt 7), retrying...
[!] GDB not ready (attempt 8), retrying...
[!] GDB not ready (attempt 9), retrying...
[!] GDB not ready (attempt 10), retrying...
[-] Could not connect to QEMU GDB stub. Exiting.
(qemu-fault-env) kazumin@PR3D4T0R:~$
```

Figure 1: QEMU termination following the first instruction corruption injection. The GDB stub signals a fatal execution failure, preventing further iterations and halting the virtual machine.

5. Discussion

The results confirm that fault injection at the register level, specifically bit flips and stuck-at faults, did not disrupt the execution of the virtual RISC-V system. All test iterations completed without a system crash or noticeable behavioral anomalies. In stark contrast, the first instruction corruption attempt triggered an immediate and unrecoverable system failure, halting QEMU and preventing subsequent injections.

Although this report does not aim to analyze detailed system behavior or recovery pathways, the observed outcomes underscore a fundamental difference in how various fault types propagate through system execution. Instruction memory corruption, particularly in kernel space, appears highly volatile and sensitive, often leading to catastrophic failure. Such behavior emphasizes the critical nature of protecting executable memory regions in fault-tolerant system designs.

6. Conclusion

This project shows how faults can be injected into a virtual RISC-V system using QEMU. It uses three fault types: bit flips, stuck-at faults, and instruction corruption. Each one simulates real hardware problems that may occur in embedded systems. Python scripts inject the faults by controlling GDB in the background. These scripts change CPU registers or instructions during runtime. The process works without needing physical hardware. That makes it easier to test systems repeatedly in a controlled way. All fault events are logged and stored in structured DOCX reports. These reports make it simple to track what went wrong and when. The results show that this method works reliably. It also helps developers test how their systems behave under stress. While this study only checks how faults are injected and recorded, it opens many doors. Future research can explore how systems handle these faults or recover from them. Overall, this framework can help improve the safety and reliability of embedded software.

Annex

The detailed fault injection logs and the complete Python scripts used for the experiments are compiled in a separate supplementary PDF document. This annex file provides comprehensive data for reproducibility and further analysis. Readers interested in the raw logs and source code may refer to this supplementary document to verify and extend the results presented in this report.

Bibliography

- [1] P. Koopman, “Understanding Fault Injection,” *IEEE Security & Privacy*, vol. 14, no. 4, pp. 71–75, Jul.–Aug. 2016.
- [2] D. Li, Z. Zhou, and X. Chen, Edge AI: On-demand accelerating deep neural network inference via edge computing, *IEEE Transactions on Wireless Communications*, vol. 19, no. 1, pp. 447457, Jan. 2020.
- [3] F. Bellard, “QEMU, a Fast and Portable Dynamic Translator,” in *USENIX Annual Technical Conference*, 2005.
- [4] J. F. Ziegler and W. A. Lanford, “Effect of cosmic rays on computer memories,” *Science*, vol. 206, no. 4420, pp. 776–788, Nov. 1979.
- [5] M. Abramovici, M. A. Breuer, and A. D. Friedman, *Digital Systems Testing and Testable Design*, Rockville, MD, USA: Computer Science Press, 1994.
- [6] H. Qi, et al., “Study on the intermittent fault mechanism of electromagnetic relay,” *Microelectronics Reliability*, vol. 54, no. 9–10, pp. 1920–1924, Sep.–Oct. 2014.
- [7] E. Roberts, “Pipelining,” Stanford University, 1999. [Online]. Available: <https://cs.stanford.edu/people/eroberts/courses/soco/projects/risc/pipelining/index.html>
- [8] M. Koppenstein, “Fast user-mode fault injection with FAIL*,” M.S. thesis, Dept. Computer Science, TU Dortmund Univ., Dortmund, Germany, 2020.